

for one row x_i :

$$y_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

each output is positive, and the row sums to approx. 1.

→ the direct formula is unsafe about "overflows in float32"

stable softmax first subtracts the row max:

$$m = \max_j x_j$$
$$y_i = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}$$

1. find the max
2. compute exp. and their sum
3. div. each exp. by their sum.

Ex: $[1000, 1001, 1002]$

$$= [1000 - 1002, 1001 - 1002, 1002 - 1002]$$

$$= [-2, -1, 0]$$

$$e^x = [0.1353, 0.3679, 1]$$

$$\text{denom} = 0.1353 + 0.3679 + 1 = 1.5032$$

$$\text{softmax} = [0.0500, 0.2447, 0.6672]$$

- Final Summary -

1. Naïve MULTI-kernel

row-max kernel

↓ global barrier

exponential kernel → normal buffer

↓ global barrier

row-sum kernel

↓ global barrier

normalization kernel

Separate launches make dependencies simple and globally synchronized.

The price is:

launch overhead,
extra global-mem traffic,
full temp. buffers

2. One-block per row

256-thread local max

↓

shmem max reduction

↓

exp + thread local sum

↓

normalize

fuses the ^{whole} operation into one launch

↳ temp. writes numerators into 'output'
then reads and overwrites them during normalization

3. One warp per row

32 lane - local max → shuffle max

32 lane - local sum → shuffle sum

Values move directly between lane registers
Removing

↳ shmem and block-wide barriers

Giving good short-row performance

4. Vectorized Block

float 4 = $\{x, y, z, w\}$

↳ uses block algo.
but handles 4 adjacents
per memory op.

→ no guarantee faster-ness
The math. workload
↳ 4 exp., reductions, and
synchronization
still exists

→ when cols % 4 != 0
launcher falls back to
scalar block softmax

